



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика и системы управления

КАФЕДРА Программное обеспечение ЭВМ и информационные технологии

КУРСОВАЯ РАБОТА

НА ТЕМУ:

Разработка базы данных приюта для животных

Студент

ИУ7-66Б

(группа)

(подпись, дата)

Барченко В.Б.

(И.О.Фамилия)

Руководитель курсового проекта

(подпись, дата)

Кострицкий А.С.

(И.О.Фамилия)

2024 г.

РЕФЕРАТ

Расчетно-пояснительная записка к курсовой работе содержит 13 иллюстраций, 10 таблиц, 2 приложения, 19 использованных источников.

Ключевые слова: база данных, система управления базами данных, реляционная модель, приют для животных, приложение, PostgreSQL, тестирование, индексация.

Объектом разработки является база данных приюта для животных.

Цель курсовой работы — разработка и реализация информационной системы, прикладная программа которой использует данные, хранящиеся в базе данных, для автоматизации деятельности приюта для животных.

Результатом работы являются разработанная база данных и приложение доступа к ней. Приложение написано на языке программирования C# с использованием паттерна MVVM и предоставляет возможности просмотра, редактирования, добавления и удаления данных в зависимости от роли пользователя. В качестве системы управления базами данных используется PostgreSQL.

Система предназначена для использования в приютах для животных, облегчая управление данными о животных, сотрудниках, опекунах и медицинских картах.

Прогнозные предположения о развитии объекта разработки:

1. Расширение функциональности приложения приюта для его использования за пределами учреждения.
2. Разработка управления финансами, волонтерской деятельностью и образовательными программами.

СОДЕРЖАНИЕ

РЕФЕРАТ	3
ВВЕДЕНИЕ	5
1 Аналитический раздел	6
1.1 Анализ предметной области	6
1.2 Анализ существующих решений	7
1.3 Типы пользователей приложения	8
1.4 Моделирование сущностей базы данных	8
1.5 Анализ типов баз данных	9
2 Конструкторский раздел	12
2.1 Сущности проектируемой базы данных	12
2.2 Проектирование функций	14
2.3 Ролевая модель	17
3 Технологический раздел	19
3.1 Средства реализации	19
3.2 Реализация базы данных	19
3.3 Интерфейс доступа к базе данных	24
3.4 Тестирование	28
4 Исследовательский раздел	30
4.1 Технические характеристики	30
4.2 Описание исследования	30
ЗАКЛЮЧЕНИЕ	34
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	35

ВВЕДЕНИЕ

В современном мире проблема бездомных животных приобретает все более острый характер. Результаты исследования проекта The State of Pet Homelessness Project показали, что в 20 странах 143 миллиона собак и 203 миллиона кошек живут на улице [1]. Одно из решений данной проблемы — создание приютов для их содержания.

Актуальность данной работы обусловлена повсеместным внедрением информационных систем для улучшения работы организаций [2], что позволяет оптимизировать управление приютом и ускорить поиск новых домов для животных.

Цель курсовой работы — разработка и реализация информационной системы, прикладная программа которой использует данные, хранящиеся в базе данных, для автоматизации деятельности приюта для животных.

Для достижения поставленной цели требуется решить следующие задачи:

- описать предметную область приюта для животных;
- сформулировать требования и ограничения к базе данных и приложению, описание пользователей проектируемого приложения;
- спроектировать сущности базы данных, ограничения целостности и ролевую модель на уровне базы данных;
- разработать спроектированное программное обеспечение;
- описать методы тестирования разработанного функционала и разработать тесты для проверки корректности работы приложения;
- провести исследование зависимости времени выполнения запроса от наличия индексации.

1 Аналитический раздел

В данном разделе будет описана предметная область, существующие решения, сформулированы требования и ограничения к разрабатываемой базе данных и приложению, описаны пользователи проектируемого приложения, а также модели баз данных.

1.1 Анализ предметной области

В 2021 году в России было зафиксировано 4,1 миллиона безнадзорных животных [3]. Результаты исследования «Бездомные животные в России» [4] выявили, что 62% опрошенных считают, что решением проблемы является убежище для животных в приютах и поиск новых домов для них. Однако лишь 19% опрошенных хорошо информированы о деятельности таких учреждений и благотворительных фондов. Это позволяет сделать вывод о необходимости не только увеличения числа приютов, но и создания более эффективного взаимодействия между приютами и благотворителями.

Деятельность приютов основана на спасении и защите животных, которые оказались на улице. В приютах животным предоставляется временное убежище, медицинская помощь, кормление и уход. Одной из основных целей приютов является поиск ответственных хозяев для животных через процесс усыновления.

Взаимодействие с приютом для животных может происходить несколькими способами. Помимо усыновления, люди могут стать волонтерами и помогать в уходе за животными, проведении прогулок и социализации. Также возможно оказание финансовой поддержки (регулярными или разовыми платежами), пожертвования кормов и медикаментов, участие в благотворительных мероприятиях, проводимых приютом.

Создание приложения приюта для животных может повысить эффективность его работы: поможет улучшить координацию работы сотрудников, позволит оформлять желающим опеку над питомцами, а база данных обеспечит необходимой информацией о животных для потенциальных усыновителей.

1.2 Анализ существующих решений

В рамках анализа было решено сосредоточить внимание на приютах для животных г. Москвы: «Домашний», «Искра» и «Муркоша».

Критерии отбирались с учетом их потенциальной значимости для пользователя.

Список критериев для сравнения выбранных решений:

1. Подробные профили животных с описанием и информацией о здоровье;
2. Возможность поиска животных по конкретным критериям (вид, порода);
3. Возможность оформления опекуна.

В процессе выбора питомца из приюта необходимо предоставить будущему владельцу подробную информацию, в том числе касающуюся здоровья животного. Система фильтрации в поиске позволит находить подходящих животных. Процедура регистрации в качестве опекуна способствует налаживанию взаимоотношений между приютом и новым хозяином, что является фундаментом для успешного усыновления.

В таблице 1.1 приведены результаты анализа.

Таблица 1.1 – Сравнение существующих решений

Решение	1	2	3
«Домашний»	–/+	–	–
«Искра»	+/-	–	+
«Муркоша»	+	+/-	–

По итогам проведенного анализа выяснилось, что ни одно из рассмотренных решений не удовлетворяет всем критериям. Разрабатываемое в данной работе программное обеспечение будет покрывать все указанные функциональные потребности.

1.3 Типы пользователей приложения

Для организации безопасности данных важно обеспечить клиентам базы данных доступ только к тем ресурсам, которые помогут решить их задачи [5]. В связи с чем были выделены три типа пользователей: администратор, ассистент и ветеринар. На рисунке 1.1 представлены их возможности.

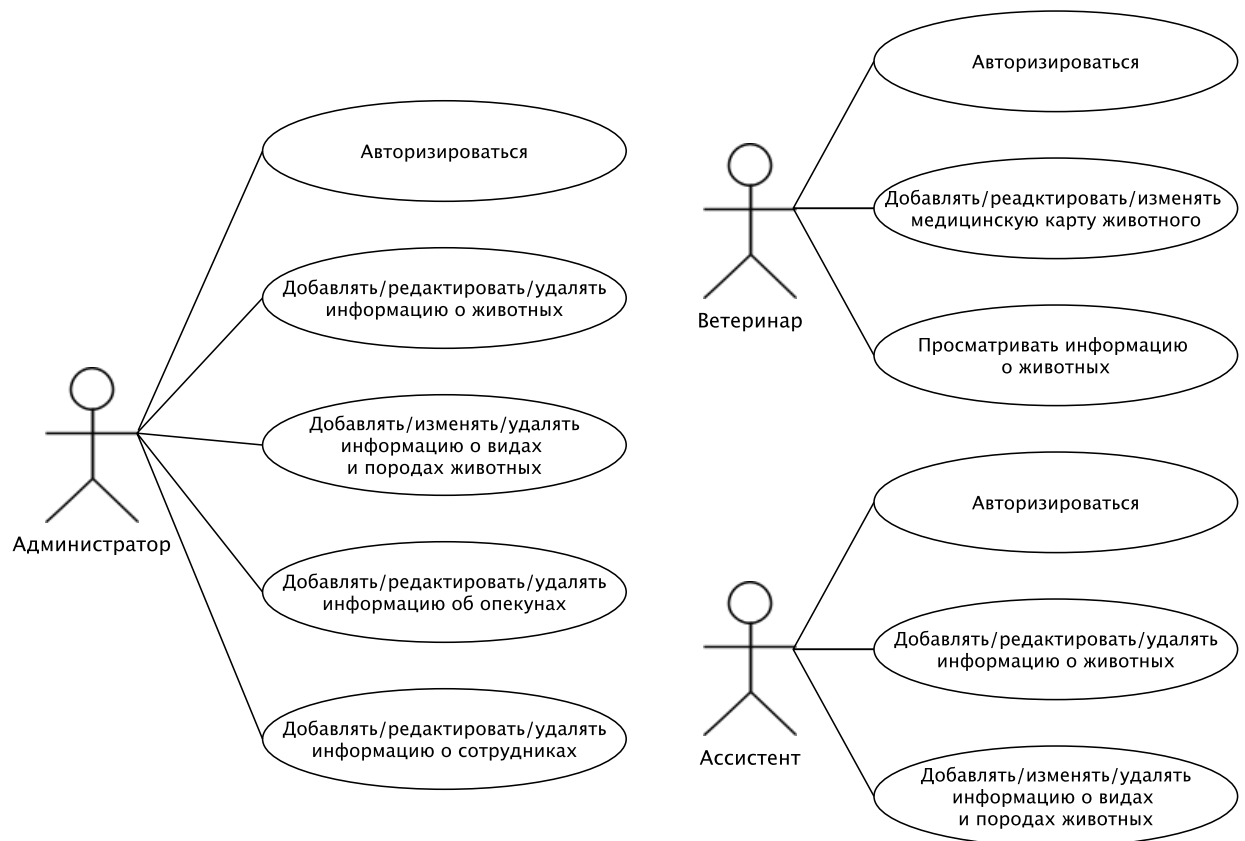


Рисунок 1.1 – Диаграмма вариантов использования

1.4 Моделирование сущностей базы данных

Для упрощения взаимопонимания между специалистом предметной области и разработчиком информационной системы выделяются сущности, атрибуты и связи [5].

Разрабатываемая база данных должна хранить всю необходимую информацию для управления приютом для животных: информацию о животных, сотрудниках, опекунах.

На рисунке 1.2 отображена ER-диаграмма сущностей базы данных в нотации Чена.

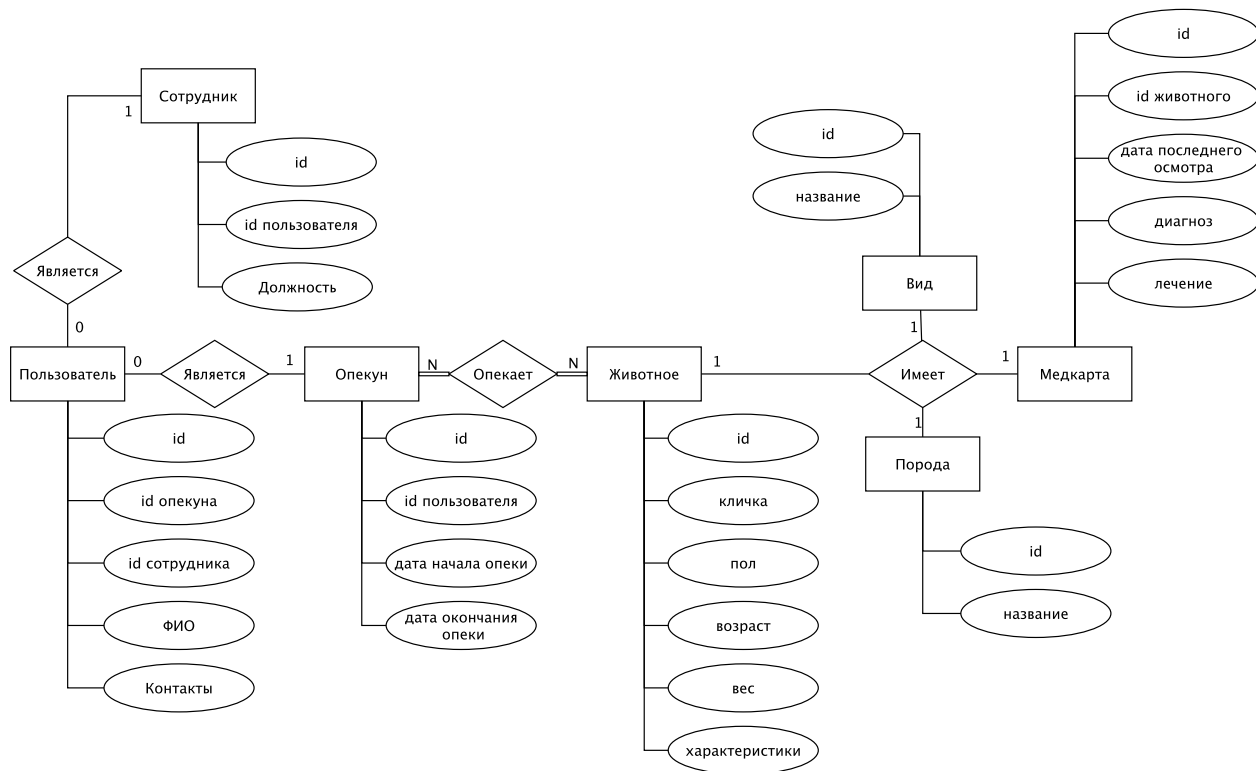


Рисунок 1.2 – Диаграмма сущностей в нотации Чена

1.5 Анализ типов баз данных

Модель данных — это совокупность правил порождения структур данных в базе данных, операций над ними, а также ограничений целостности, определяющих допустимые связи и значения данных, последовательность их изменения [6]. Модель данных является инструментом моделирования произвольной предметной области [7]. Можно выделить следующие модели баз данных [8]:

- иерархические;
- сетевые;
- реляционные;

- объектно-ориентированные;
- постреляционные;
- многомерные.

Рассмотрим некоторые из них. *Иерархическая модель* представляет собой древовидный граф с записями в виде узлов, соединенных связями, называемых ребрами [8]. Все узлы дерева, за исключением корневого (самого верхнего узла, являющегося точкой входа) должны иметь родительский узел. Достоинствами данной модели являются эффективное использование памяти и удобство при работе с иерархически упорядоченной информацией, недостатком — громоздкость для обработки информации со сложными логическими связями [9].

Сетевая модель обобщает иерархическую и отображает данные в виде произвольного графа. Состоит из набора записей и набора соответствующих связей. Сетевая модель дает возможность эффективной реализации по памяти. Недостаток состоит в ослабленности контроля целостности связей из-за допустимости произвольных установлений связей между записями [9].

Для *реляционной модели* разработан теоретический базис с использованием теории множеств [7]. Базовой структурой является отношение, представляющее двумерную таблицу, содержащую данные. Объект, данные о котором хранятся в базе данных, называется сущностью. Информация о сущности хранится в отношении. Атрибуты представляют собой характеристики сущности. Каждый атрибут в структуре таблицы именуется и соответствует заголовку определенного столбца. Для описания сложных логических структур данных из предметной области используется связывание таблиц [9]. Таблицы должны быть нормализованы, что позволяет устранить дублирование данных, обеспечить их непротиворечивость [10]. Недостатки реляционной модели заключаются в относительно низкой скорости доступа и большом объеме внешней памяти, а также в сложности описания иерархических и сетевых связей [9].

Сетевая и иерархические модели в контексте базы данных приюта для животных затрудняют обработку информации предметной области. Предпочтительным является использование реляционной модели.

Вывод

В данном разделе была описана предметная область приюта для животных, были рассмотрены существующие решения, описаны требования и ограничения к разрабатываемой базе данных и приложению, пользователи проектируемого приложения, а также модели баз данных. Для представления данных была выбрана реляционная модель.

2 Конструкторский раздел

В данном разделе будут описаны сущности проектируемой базы данных, ограничения целостности, функции и ролевая модель.

2.1 Сущности проектируемой базы данных

На рисунке 2.1 изображены сущности проектируемой базы данных в соответствии с рисунком 1.2.

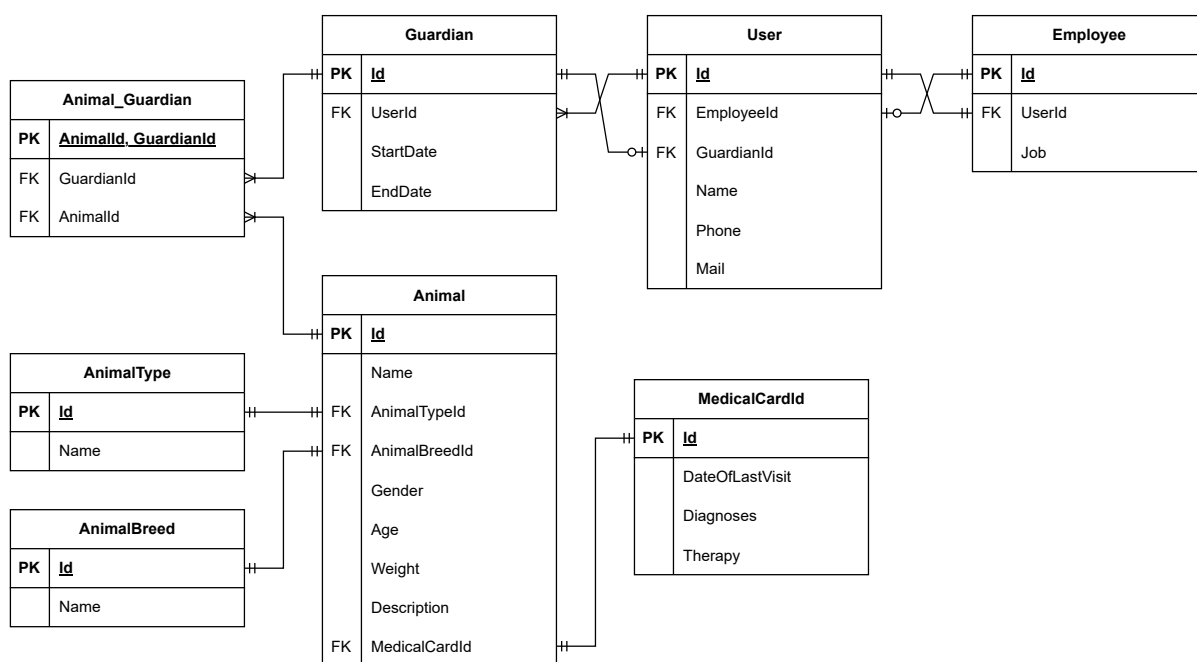


Рисунок 2.1 – Диаграмма базы данных

Описание сущностей Animal, MedicalCard, AnimalType и AnimalBreed приведено соответственно в таблицах 2.1 — 2.4.

Таблица 2.1 – Описание сущности Animal

Атрибут	Тип данных	Ограничения
Id	SERIAL	PK
Name	TEXT	NOT NULL
AnimalTypeId	INT	FK (AnimalType.Id)
AnimalBreedId	INT	FK (AnimalBreed.Id)
Gender	INT	NOT NULL
Age	INT	NOT NULL
Weight	DOUBLE PRECISION	NOT NULL
Description	TEXT	NOT NULL
MedicalCardId	INT	FK (MedicalCard.Id)

Таблица 2.2 – Описание сущности MedicalCard

Атрибут	Тип данных	Ограничения
Id	SERIAL	PK
AnimalId	INT	FK (Animal.Id)
DateOfLastVisit	TIMESTAMP	NOT NULL
Diagnoses	TEXT	NOT NULL
Therapy	TEXT	NOT NULL

Таблица 2.3 – Описание сущности AnimalType

Атрибут	Тип данных	Ограничения
Id	SERIAL	PK
Name	TEXT	NOT NULL

Таблица 2.4 – Описание сущности AnimalBreed

Атрибут	Тип данных	Ограничения
Id	SERIAL	PK
Name	TEXT	NOT NULL

Промежуточная сущность Animal_Guardian реализует связь многие-ко-многим между животными и их опекунами. Это позволяет животному иметь несколько опекунов, а каждый опекун может заботиться о нескольких питомцах.

Таблица 2.5 – Описание сущности Animal_Guardian

Атрибут	Тип данных	Ограничения
AnimalId	INT	FK (Animal.Id), PK
GuardianId	INT	FK (Guardian.Id), PK

Таблица ShelterUser представляет собой универсальную сущность, которая может быть связана как с сотрудниками приюта, так и с опекунами животных. В таблицах 2.6 — 2.8 приведено описание сущностей ShelterUser, Guardian и Employee соответственно.

Таблица 2.6 – Описание сущности ShelterUser

Атрибут	Тип данных	Ограничения
Id	SERIAL	PK
IdEmployee	INT	FK (Employee.Id)
GuardianId	INT	FK (Guardian.Id)
Name	TEXT	NOT NULL
Phone	TEXT	NOT NULL
Mail	TEXT	NOT NULL

Таблица 2.7 – Описание сущности Guardian

Атрибут	Тип данных	Ограничения
Id	SERIAL	PK
UserId	INT	
StartDate	DATE	NOT NULL
EndDate	DATE	

Таблица 2.8 – Описание сущности Employee

Атрибут	Тип данных	Ограничения
Id	SERIAL	PK
UserId	INT	
Job	TEXT	NOT NULL

2.2 Проектирование функций

Для обеспечения удобства работы с базой данных приюта животных, будут спроектированы функции-фильтры. Эти инструменты позволят поль-

зователям системы эффективно сортировать и извлекать информацию о питомцах, используя критерии, такие как вид и порода животного.

На рисунках 2.2 – 2.4 приведены схемы функций-фильтров соответственно по виду, породе, виду и породе одновременно.

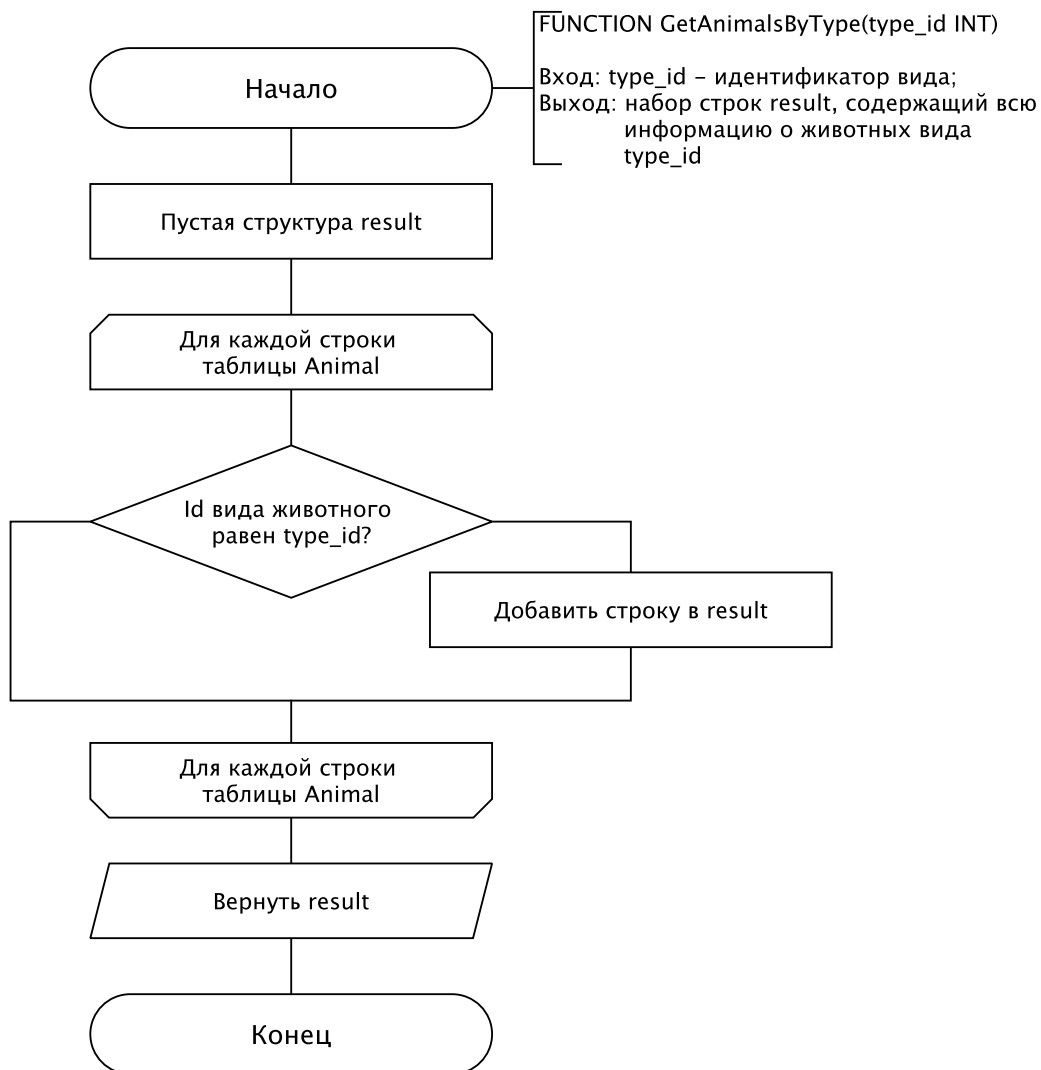


Рисунок 2.2 – Схема функции получения животных определенного вида

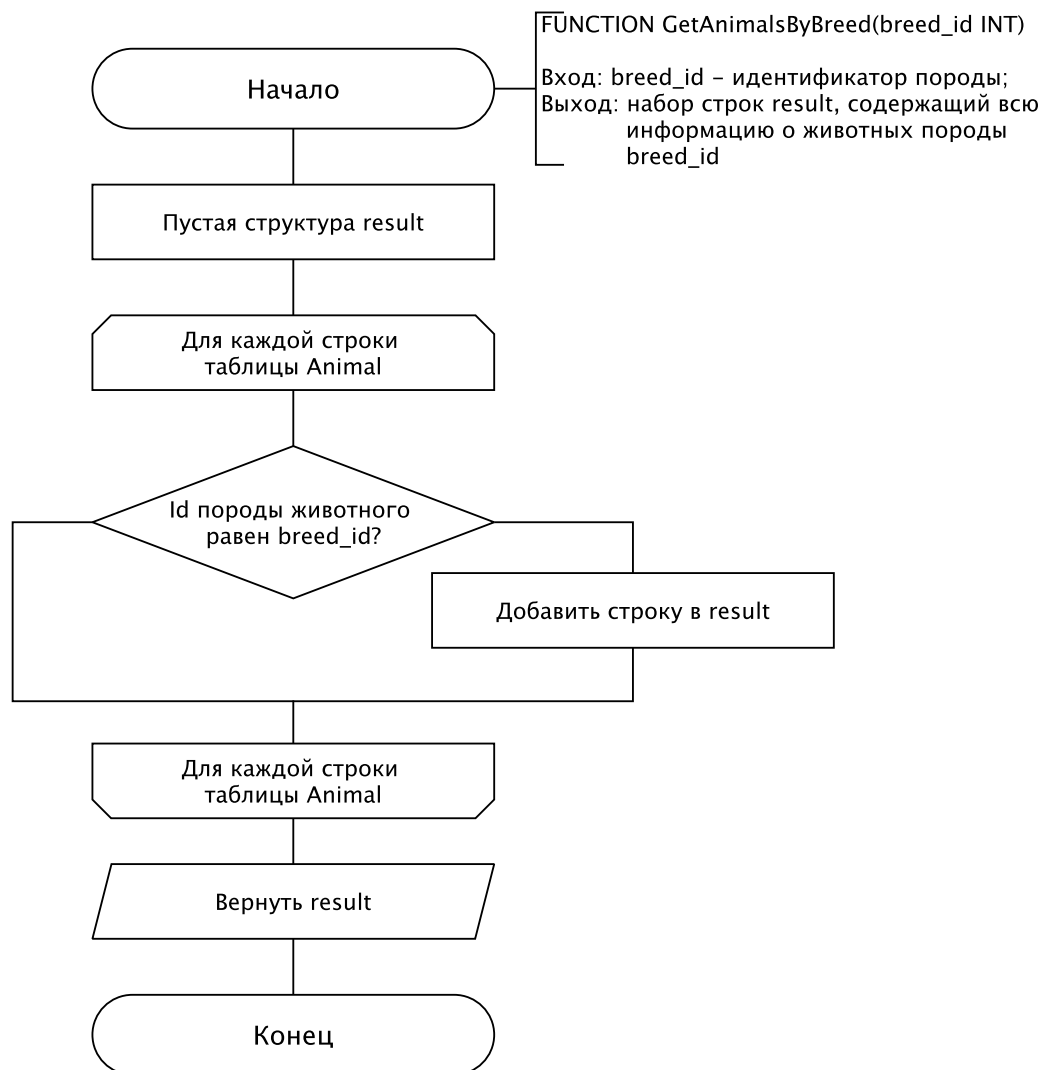


Рисунок 2.3 – Схема функции получения животных определенной породы

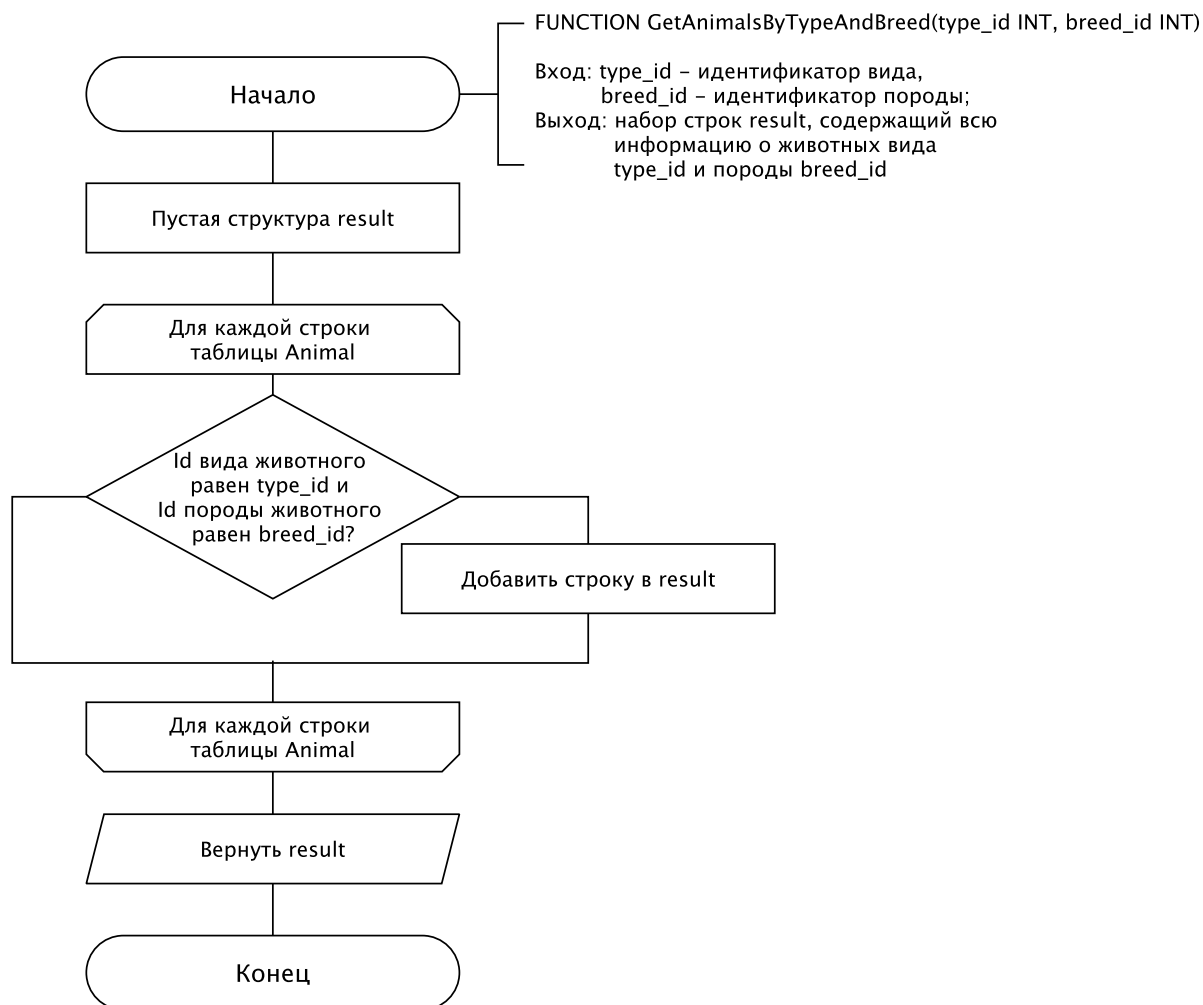


Рисунок 2.4 – Схема функции получения животных определенного вида и породы

2.3 Ролевая модель

Опираясь на описанные типы пользователей на рисунке 1.1, выделим следующую ролевую модель:

1. Администратор имеет полный доступ ко всем таблицам, включая права на вставку, обновление, удаление и выборку данных;
2. Ассистент имеет права на обновление, удаление, вставку и выборку данных в таблицах Animal, AnimalType, AnimalBreed, Guardian, Shelteruser и Animal_Guardian, а также право на выборку информации из таблиц Employee и MedicalCard;

3. Ветеринар имеет право на выборку данных из таблиц Animal, Employee, AnimalType, AnimalBreed, Animal_Guardian и Guardian. Может обновлять, удалять, вставлять и выбирать данные в таблице MedicalCard.

Вывод

В данном разделе были описаны сущности проектируемой базы данных, ограничения целостности, схемы процедур и ролевая модель.

3 Технологический раздел

В данном разделе будут описаны средства реализации, тестирование, интерфейс доступа к базе данных, приведены реализации создания таблиц, ролей и функций.

3.1 Средства реализации

Для реализации приложения доступа к базе данных был выбран язык программирования C# [13]. Данный язык имеет все необходимые и достаточные инструменты для решения поставленной задачи: поддержка парадигмы объектно-ориентированного программирования даст возможность создавать структуру программы с классами и объектами, соответствующими спроектированным сущностям. Для графического интерфейса выбран фреймворк Avalonia [14].

Исходя из выбора реляционной модели базы данных в разделе 1.5 в качестве системы управления базами данных выбрана PostgreSQL [15]. Она совместима с ACID-транзакциями, что гарантирует надежность данных. Система обеспечивает высокую производительность. Она полностью совместима со стандартами SQL, что облегчает разработку, а также подробно документирована [16].

Для создания соединений и выполнения SQL-запросов использована библиотека Npgsql, которая является .NET провайдером для PostgreSQL. Она позволяет разработчикам .NET подключаться к базам данных PostgreSQL, выполнять запросы и обрабатывать результаты. Npgsql поддерживает все современные функции PostgreSQL и полностью совместима со стандартами SQL [17].

3.2 Реализация базы данных

В листинге 3.6 представлено создание таблиц базы данных в соответствии с диаграммой 2.1.

Листинг 3.1 – Создание таблиц базы данных

```
1 CREATE TABLE MedicalCard(  
2     Id SERIAL PRIMARY KEY,  
3     AnimalId INT,  
4     DateOfLastVisit TIMESTAMP NOT NULL,  
5     Diagnoses TEXT NOT NULL,  
6     Therapy TEXT NOT NULL  
7 );  
8  
9 CREATE TABLE AnimalType(  
10     Id SERIAL PRIMARY KEY,  
11     Name TEXT NOT NULL  
12 );  
13  
14 CREATE TABLE AnimalBreed(  
15     Id SERIAL PRIMARY KEY,  
16     Name TEXT NOT NULL  
17 );  
18  
19 CREATE TABLE Animal(  
20     Id SERIAL PRIMARY KEY,  
21     Name TEXT NOT NULL,  
22     AnimalTypeId INT REFERENCES AnimalType(Id) ON DELETE CASCADE,  
23     AnimalBreedId INT REFERENCES AnimalBreed(Id) ON DELETE CASCADE,  
24     Gender INT NOT NULL,  
25     Age INT NOT NULL,  
26     Weight DOUBLE PRECISION NOT NULL,  
27     Description TEXT NOT NULL,  
28     MedicalCardId INT REFERENCES MedicalCard(Id) ON DELETE SET NULL  
29 );  
30  
31 ALTER TABLE MedicalCard  
32 ADD FOREIGN KEY (AnimalId) REFERENCES Animal(Id) ON DELETE CASCADE;  
33  
34 CREATE TABLE Guardian(  
35     Id SERIAL PRIMARY KEY,  
36     UserId INT,  
37     StartDate DATE NOT NULL,  
38     EndDate DATE  
39 );  
40  
41 CREATE TABLE Animal_Guardian(  
42     AnimalId INT REFERENCES Animal(Id) ON DELETE CASCADE,  
43     GuardianId INT REFERENCES Guardian(Id) ON DELETE CASCADE,  
44     PRIMARY KEY (AnimalId, GuardianId)  
45 );
```

Листинг 3.2 – Создание таблиц базы данных (продолжение)

```
1 CREATE TABLE Employee(  
2     Id SERIAL PRIMARY KEY,  
3     UserId INT,  
4     Job TEXT NOT NULL  
5 );  
6  
7 CREATE TABLE ShelterUser(  
8     Id SERIAL PRIMARY KEY,  
9     IdEmployee INT REFERENCES Employee(Id),  
10    GuardianId INT REFERENCES Guardian(Id),  
11    Name TEXT NOT NULL,  
12    Phone TEXT NOT NULL,  
13    Mail TEXT NOT NULL  
14 );  
15  
16 ALTER TABLE Employee  
17 ADD FOREIGN KEY (UserId) REFERENCES ShelterUser(Id) ON DELETE CASCADE;  
18 ALTER TABLE Guardian  
19 ADD FOREIGN KEY (UserId) REFERENCES ShelterUser(Id) ON DELETE CASCADE;
```

В листинге 3.3 приведено создание ролей базы данных, спроектированных в пункте 2.3.

Листинг 3.3 – Создание ролей базы данных

```
1 CREATE ROLE administrator WITH LOGIN PASSWORD 'admin_pswrd';  
2 GRANT INSERT, UPDATE, DELETE, SELECT ON ALL TABLES IN SCHEMA public TO  
3     administrator;  
4 GRANT USAGE, SELECT ON SEQUENCE medicalcard_id_seq,  
5     animaltype_id_seq,  
6     animalbreed_id_seq,  
7     animal_id_seq,  
8     guardian_id_seq,  
9     employee_id_seq,  
10    shelteruser_id_seq TO administrator;  
11  
12 CREATE ROLE assistant WITH LOGIN PASSWORD 'assistant_pswrd';  
13 GRANT UPDATE, DELETE, INSERT, SELECT ON Animal, AnimalType, AnimalBreed,  
14     Guardian, Shelteruser,  
15     Animal_Guardian TO assistant;
```

Листинг 3.4 – Создание ролей базы данных (продолжение)

```
1 GRANT SELECT ON Employee, MedicalCard TO assistant;
2 GRANT INSERT ON MedicalCard TO assistant;
3 GRANT USAGE, SELECT ON SEQUENCE animal_id_seq,
4                                     medicalcard_id_seq,
5                                     guardian_id_seq,
6                                     animaltype_id_seq,
7                                     shelteruser_id_seq,
8                                     animalbreed_id_seq TO assistant;
9
10
11 CREATE ROLE vet WITH LOGIN PASSWORD 'vet_pswrd';
12 GRANT SELECT ON Animal, Employee, AnimalType, AnimalBreed,
13             Animal_Guardian, Guardian TO vet;
14 GRANT UPDATE, DELETE, INSERT, SELECT ON MedicalCard TO vet;
```

В листингах 3.5 — 3.8 приведены реализации функций получения выборки животных определенного вида и породы, спроектированные в разделе 2.2.

Листинг 3.5 – Реализация функции получения животных определенной породы

```
1 CREATE OR REPLACE FUNCTION GetAnimalsByBreed(breed_id INT)
2 RETURNS TABLE (
3     animal_id INT,
4     name TEXT,
5     animal_type_id INT,
6     animal_breed_id INT,
7     gender INT,
8     age INT,
9     weight DOUBLE PRECISION,
10    description TEXT,
11    medical_card_id INT
12 ) AS $$
13 BEGIN RETURN QUERY
14     SELECT
15         a.Id AS animal_id,
16         a.Name AS name,
17         a.AnimalTypeId AS animal_type_id,
18         a.AnimalBreedId AS animal_breed_id,
19         a.Gender AS gender,
20         a.Age AS age,
21         a.Weight AS weight,
22         a.Description AS description,
23         a.MedicalCardId AS medical_card_id
24
```

Листинг 3.6 – Реализация функции получения животных определенной породы (продолжение)

```
1      FROM Animal AS a
2      WHERE a.AnimalBreedId = breed_id;
3 END;
4 $$ LANGUAGE plpgsql;
```

Листинг 3.7 – Реализация функции получения животных определенного вида

```
1 CREATE OR REPLACE FUNCTION GetAnimalsByType(type_id INT)
2 RETURNS TABLE (
3     animal_id INT,
4     name TEXT,
5     animal_type_id INT,
6     animal_breed_id INT,
7     gender INT,
8     age INT,
9     weight DOUBLE PRECISION,
10    description TEXT,
11    medical_card_id INT
12 ) AS $$
13 BEGIN
14     RETURN QUERY
15     SELECT
16         a.Id AS animal_id,
17         a.Name AS name,
18         a.AnimalTypeId AS animal_type_id,
19         a.AnimalBreedId AS animal_breed_id,
20         a.Gender AS gender,
21         a.Age AS age,
22         a.Weight AS weight,
23         a.Description AS description,
24         a.MedicalCardId AS medical_card_id
25     FROM Animal AS a
26     WHERE a.AnimalTypeId = type_id;
27 END;
28 $$ LANGUAGE plpgsql;
```

Листинг 3.8 – Реализация функции получения животных определенного вида и породы

```
1 CREATE OR REPLACE FUNCTION GetAnimalsByTypeAndBreed(type_id INT, breed_id
  INT)
2 RETURNS TABLE (
3     animal_id INT,
4     name TEXT,
5     animal_type_id INT,
6     animal_breed_id INT,
7     gender INT,
8     age INT,
9     weight DOUBLE PRECISION,
10    description TEXT,
11    medical_card_id INT
12 ) AS $$
13 BEGIN
14     RETURN QUERY
15     SELECT
16         a.Id AS animal_id,
17         a.Name AS name,
18         a.AnimalTypeId AS animal_type_id,
19         a.AnimalBreedId AS animal_breed_id,
20         a.Gender AS gender,
21         a.Age AS age,
22         a.Weight AS weight,
23         a.Description AS description,
24         a.MedicalCardId AS medical_card_id
25     FROM Animal AS a
26     WHERE a.AnimalTypeId = type_id and a.AnimalBreedId = breed_id;
27 END;
28 $$ LANGUAGE plpgsql;
```

3.3 Интерфейс доступа к базе данных

Для обеспечения доступа к базе данных и её управления было создано приложение, используя паттерн Model-View-ViewModel. Данный паттерн разделяет программное решение на 3 компонента [19]:

1. Модель (Model) – играет роль описания используемых данных.
2. Представление (View) – интерфейс, с которым пользователь ведет взаимодействие;

3. Модель представления (ViewModel) – компонент, выполняющий роль связующего звена между моделями программы и их представлением.

На рисунках 3.1 – 3.5 изображены интерфейсы окон приложения.

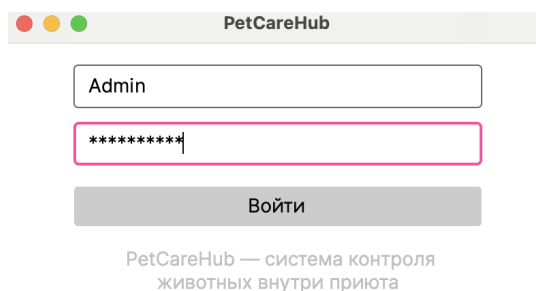


Рисунок 3.1 – Окно авторизации

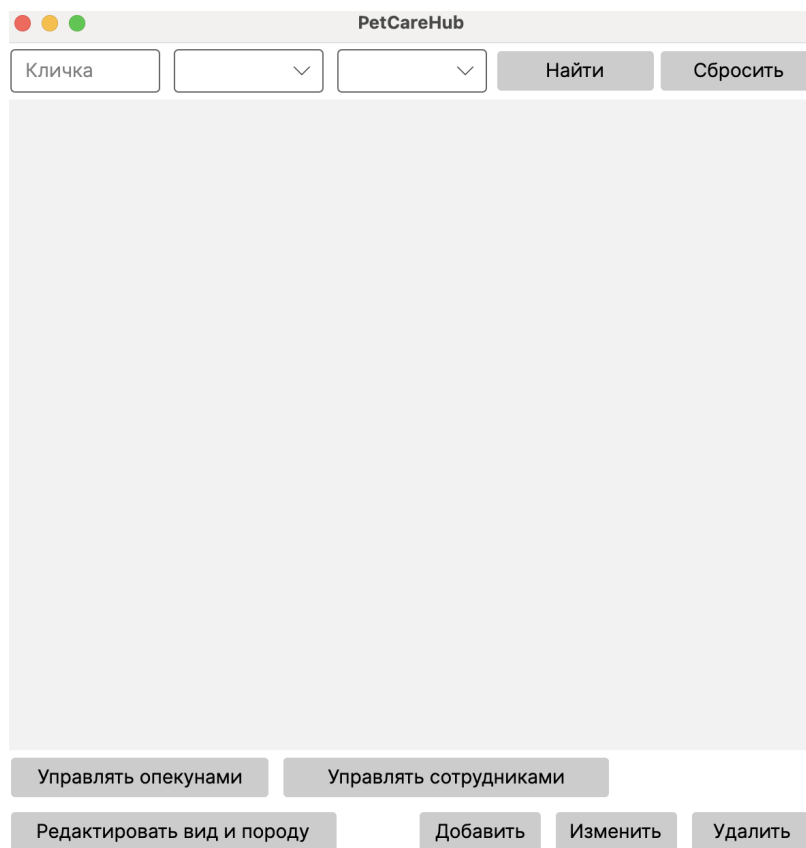


Рисунок 3.2 – Главное окно

Рисунок 3.4 – Окно редактирования информации о сотрудниках

Управление опекунами

Имя:

Телефон:

Почта:

Дата начала опеки:

Дата окончания опеки:

Сохранить Изменить

Очистить Добавить Удалить

Рисунок 3.5 – Окно редактирования информации об опекунах

3.4 Тестирование

Тестирование по методу черного ящика для функций базы данных было написано с использованием библиотеки NUnit. Создание и удаление тестовой базы данных также автоматизированы с помощью методов `SetUp` и `TearDown`. Листинг тестовых случаев для одной из функций приведен в Приложении Б. Все тесты успешно пройдены (рисунок 3.6).

```

leurok@leurok-2 ~/Study/DB_Course_work/AnimalShelter/AnimalShelterTests$ dotnet test
Определение проектов для восстановления...
Все проекты обновлены для восстановления.
AnimalShelterCore -> /Users/leurok/Study/DB_Course_work/AnimalShelter/AnimalShelterCore/bin/Debug/net7.0/AnimalShelterCore.dll
AnimalShelterPostgreSql -> /Users/leurok/Study/DB_Course_work/AnimalShelter/AnimalShelterPostgreSql/bin/Debug/net7.0/AnimalShelterPostgreSql.dll
AnimalShelterTests -> /Users/leurok/Study/DB_Course_work/AnimalShelter/AnimalShelterTests/bin/Debug/net7.0/AnimalShelterTests.dll
Тестовый запуск для /Users/leurok/Study/DB_Course_work/AnimalShelter/AnimalShelterTests/bin/Debug/net7.0/AnimalShelterTests.dll (.NET CoreApp,Version=v7.0)
Программа Microsoft (R) Test Execution Command Line Tool версии 17.5.0 (arm64)
(с) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Запуск выполнения тестов; подождите...
Общее количество тестовых файлов (1), соответствующих указанному шаблону.

Пройден!      : не пройдено      0, пройдено      12, пропущено      0, всего      12, длительность 1 s. - AnimalShelterTests.dll (net7.0)

```

Рисунок 3.6 – Результат тестирования разработанных на стороне базы данных функций

Вывод

В данном разделе были описаны средства реализации, тестирование, интерфейс доступа к базе данных, приведены реализации создания таблиц, ролей и функций.

4 Исследовательский раздел

В данном разделе будут описаны технические характеристики устройства, цель и описание исследования, приведены его результаты, а также на их основе будут сделаны выводы.

4.1 Технические характеристики

Технические характеристики устройства, на котором выполнялось исследование:

- операционная система macOS Sonoma Версия 14.5 [11];
- оперативная память 16 ГБайт;
- процессор Apple M1 Pro [12].

При проведении тестирования внешние задачи отсутствовали.

4.2 Описание исследования

Исследовательская цель заключается в анализе влияния индексирования на время выполнения запроса в базе данных.

Индекс — это структура, которая определяет соответствие значения ключа записи (атрибута или группы атрибутов) и местоположения этой записи [18]. Индексирование используется для ускорения доступа к записям по значению ключа и не влияет на размещение данных этой таблицы.

Создание составного индекса для таблицы `Animal` приведено в листинге 4.1

Листинг 4.1 – Создание составного индекса для таблицы `Animal`

```
1 CREATE INDEX idx_animal_type_breed ON Animal(AnimalTypeId, AnimalBreedId);
```

В качестве запроса была выбрана разработанная функция получения выборки животных определенного вида и породы, текст реализации которой приведен в листинге 3.8.

В рамках исследования было проведено сравнение времени выполнения запроса двух баз данных: одной с индексированием и другой без него. Измерение времени выполнения 1000 одинаковых запросов осуществлялось на наборах тестовых данных от 1000 до 10000 записей с шагом 500. Для каждого размера выборки результаты были усреднены.

Для замеров времени использовался sql-скрипт, текст которого приведен в листинге 4.2

Листинг 4.2 – Sql-скрипт для замеров времени

```
1 CREATE OR REPLACE FUNCTION measure_time (iterations int)
2 RETURNS TABLE (iters int, totalTime interval)
3 AS $$
4 DECLARE
5     i          int;
6     startTime  TIMESTAMP;
7     endTime    TIMESTAMP;
8     tmp        text;
9 BEGIN
10    i := 1;
11    startTime := clock_timestamp();
12    WHILE ( i <= iterations) LOOP
13        SELECT * INTO tmp FROM GetAnimalsByTypeAndBreed(3, 7);
14        i := i + 1;
15    END LOOP;
16
17    endTime := clock_timestamp();
18    iters := iterations;
19    totalTime := (endTime - startTime) / iterations;
20    RETURN NEXT;
21 END;
22 $$ language plpgsql;
23 SELECT * FROM measure_time(1000);
```

Результаты замеров времени приведены в таблице 4.1.

Таблица 4.1 – Результаты замеров времени

Количество записей	Время выполнения запроса с индексацией, мс	Время выполнения запроса без индексации, мс
1000	0.018	0.097
1500	0.021	0.143
2000	0.017	0.174
2500	0.016	0.195
3000	0.020	0.230
3500	0.021	0.260
4000	0.019	0.302
4500	0.022	0.330
5000	0.020	0.376
5500	0.025	0.404
6000	0.020	0.439
6500	0.019	0.465
7000	0.018	0.502
7500	0.026	0.543
8000	0.021	0.572
8500	0.022	0.605
9000	0.025	0.636
9500	0.022	0.665
10000	0.043	0.704

Результаты замеров времени также отображены на рисунке 4.1

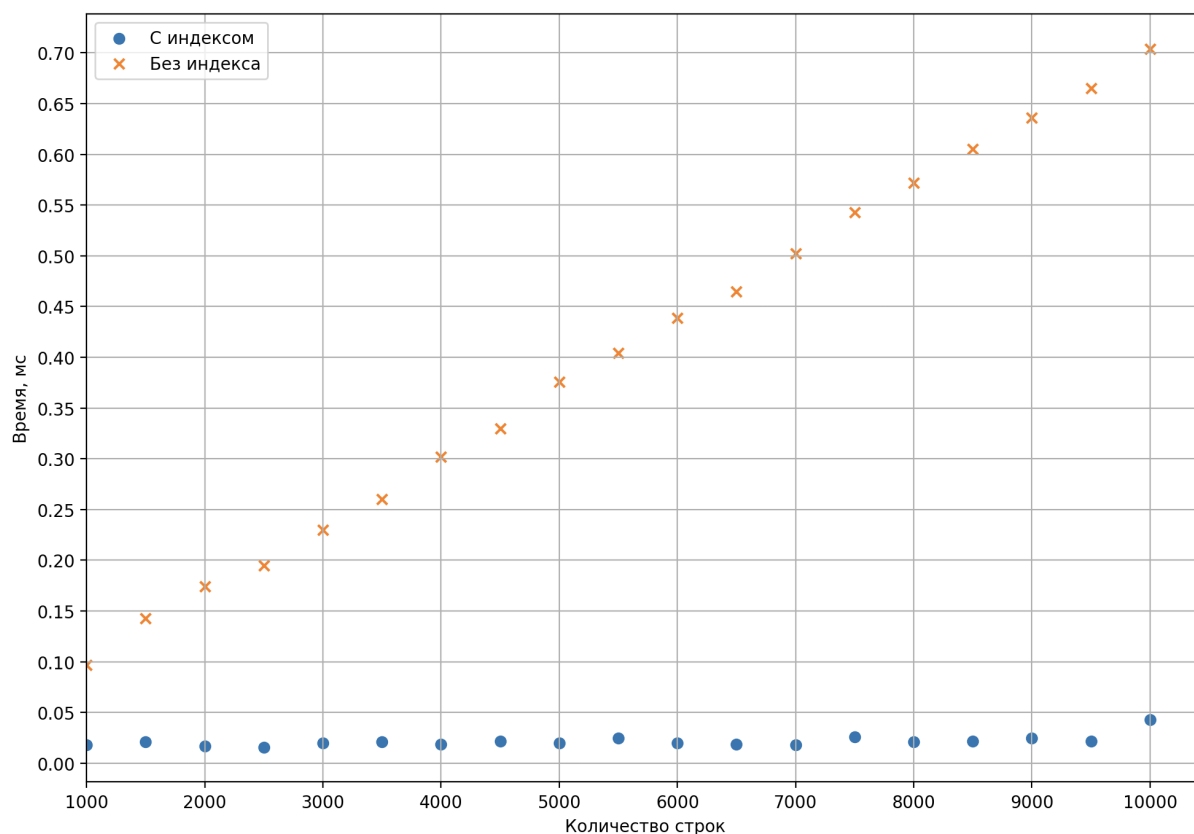


Рисунок 4.1 – Зависимость времени выполнения запроса от наличия индексации и количества записей в таблице

Вывод

Таким образом, результаты показали, что выполнение запросов при наличии индексации эффективнее по времени по сравнению с выполнением запросов без индексации. Было выявлено, что время выполнения операций с индексацией составляет 0.000391 секунды, в то время как для операций без индексации это время увеличивается до 0.007682 секунды. Индексация ускорила выполнение операций примерно в 19.64 раза. Такое улучшение производительности демонстрирует возможность преимущества использования индексов в базах данных, что может быть важным для оптимизации запросов и повышения эффективности работы с большими объемами данных.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы поставленная цель была достигнута: разработана и реализована информационная система, прикладная программа которой использует данные, хранящиеся в базе данных, для автоматизации деятельности приюта для животных.

Было проведено исследование зависимости времени выполнения запроса от наличия индексации, результат которого показал, что наличие индексации может сделать выполнение запросов эффективнее по времени.

Были решены следующие задачи:

- описана предметная область приюта для животных;
- сформулированы требования и ограничения к базе данных и приложению, описание пользователей проектируемого приложения;
- спроектированы сущности базы данных, ограничения целостности и ролевая модель на уровне базы данных;
- разработано спроектированное программное обеспечение;
- описаны методы тестирования разработанного функционала и разработаны тесты для проверки корректности работы приложения;
- проведено исследование зависимости времени выполнения запроса от наличия индексации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. STATE OF PET HOMELESSNESS PROJECT [Электронный ресурс]. — Режим доступа: <https://www.stateofpethomelessness.com/map/> (дата обращения: 07.03.2024).
2. Арсланова И. С. Автоматизированная информационная система приюта по содержанию безнадзорных (бездомных) животных / научный руководитель Слепова А. Ш. // Инновационный путь развития как ответ на вызовы нового времени: сборник статей Международной научно-практической конференции (17 сентября 2020 г, г. Челябинск). - Уфа: OMEGA SCIENCE, 2020. - 246 с.
3. ТАСС [Электронный ресурс]. — Режим доступа: <https://tass.ru/obshchestvo/12947457> (дата обращения: 07.03.2024).
4. Исследование добра mail.ru. Бездомные животные в России [Электронный ресурс]. — Режим доступа: <https://charitystats.mail.ru/pets/> (дата обращения: 07.03.2024).
5. Новиков Б. А. Основы технологий без данных. — 2-е изд. — М.: ДМК Пресс, 2020. — 582 с.
6. ГОСТ 20886-85. Организация данных в системах обработки данных. Термины и определения.
7. Карпова И.П. Базы данных. Учебное пособие. – Московский государственный институт электроники и математики (Технический университет). – М., 2009.
8. Гущин А. Н. Базы данных. 2-е изд., испр. и доп. : учебно-методическое пособие / А. Н. Гущин - М.-Берлин: Директ-Медиа, 2015. - 311 с.
9. Хомоненко А. Д., Цыганков В. М., Мальцев М. Г. Базы данных: Учебник для высших учебных заведений / Под ред. прохр. А. Д. Хомопенко. — 6-с изл., доп. — СПб: КОРОНА-Вск, 2009. — 736 с.
10. Кузин А. В. Базы данных: учеб. пособие для студ. высш. учеб. заведений. - 2-е изд., стер. -М. : Издательский центр «Академия»6 2008. - 320 с.

11. macOS Ventura - Apple(RU) [Электронный ресурс]. Режим доступа: <https://www.apple.com/macos/ventura/> (дата обращения: 01.05.2024).
12. M1 [Электронный ресурс]. Режим доступа: <https://www.apple.com/russian/newsroom/2021/10/introducing-m1-pro-and-m1-max-the-most-powerful-chips-apple-has-ever-built/> (дата обращения: 01.05.2024).
13. Документация по C#. [Электронный ресурс]. Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/csharp/> (дата обращения: 10.05.2024).
14. Avalonia UI. [Электронный ресурс]. Режим доступа: <https://www.avaloniaui.net> (дата обращения: 10.05.2024).
15. PostgreSQL: The World's Most Advanced Open Source Relational Database [Электронный ресурс]. Режим доступа: <https://www.postgresql.org> (дата обращения: 10.05.2024).
16. Джуба С., Волков А. Изучаем PostgreSQL 10 / пер. с англ. А. А. Слинкина. - М.: ДМК Пресс, 2019. - 400 с.: ил.
17. Npgsql - .NET Access to PostgreSQL [Электронный ресурс]. Режим доступа: <https://www.npgsql.org/#about> (дата обращения: 10.05.2024).
18. Карпова И.П. Базы данных. Курс лекций и материалы для практических заданий. – Учебное пособие. – М.: Питер, 2013. – 240 с.
19. Гатин Р.Р. ПРИМЕНЕНИЕ ПАТТЕРНА ПРОЕКТИРОВАНИЯ MVVM В РАЗРАБОТКЕ СОВРЕМЕННЫХ ПРОГРАММНЫХ РЕШЕНИЙ [Электронный ресурс]. Режим доступа: https://www.elibrary.ru/download/elibrary_50818776_23588387.pdf (дата обращения: 10.05.2024).